



哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY



分布式机器学习(II)

分布式同步SGD与异步SGD

王宏志

哈尔滨工业大学

计算学部海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

目录

Contents

- 1 分布式同步SGD
- 2 抗滞后同步SGD变体
- 3 分布式异步SGD
- 4 模型与流水线并行

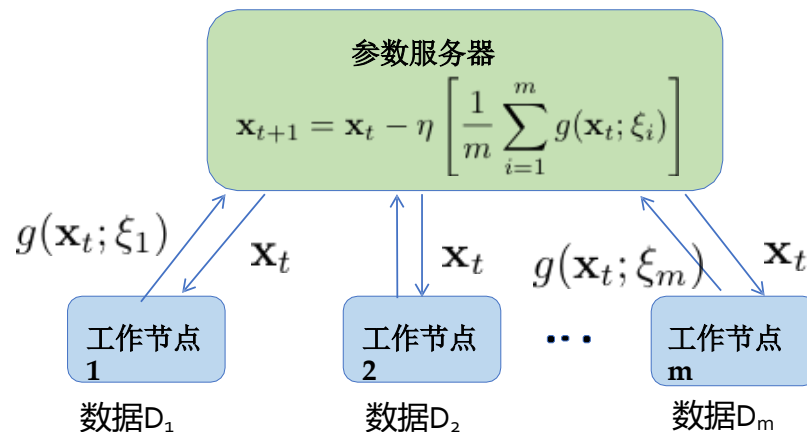
目录

Contents

- 1 分布式同步SGD
- 2 抗滞后同步SGD变体
- 3 分布式异步SGD
- 4 模型与流水线并行

为何采用分布式SGD?

- 对于大规模训练数据集，在单一节点上进行训练可能极其耗时。
- 解决方案：将数据集分割至 m 个节点上，形成分区 D_1 、 D_2 、..... D_m ，并执行数据并行分布式训练，采用一种名为同步分布式SGD的算法。



同步分布式SGD

在每次迭代中

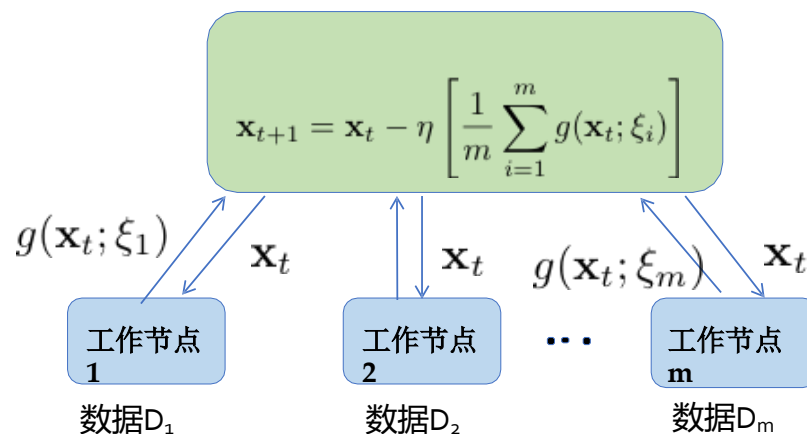
1. 参数服务器向所有 m 个工作节点发送当前参数向量 $\mathbf{x}_t$

2. 每个工作节点 i 计算梯度 $g(\mathbf{x}_t; \xi_i) = \frac{1}{b} \sum_{j=1}^b \nabla f(\mathbf{x}_t; \xi_{i,j})$

使用从分区 D_i 中均匀随机抽取的 b 个样本组成的小批量进行运算

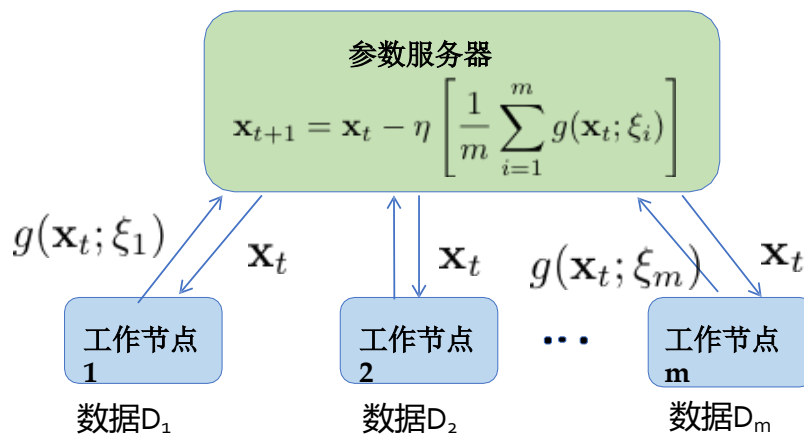
3. 参数服务器收集这 m 个小批量梯度后

按如下方式更新参数向量: $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{1}{m} \sum_{i=1}^m g(\mathbf{x}_t; \xi_i)$



同步随机梯度下降的收敛性分析

- 更新规则 $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{1}{m} \sum_{i=1}^m g(\mathbf{x}_t; \xi_i)$ 等价于
执行小批量SGD,但使用批量大小为 mb 而非仅 b 样本!
- 因此, 小批量SGD的收敛性分析可直接应用于此。
- 唯一变化的参数是方差上界。



同步随机梯度下降的收敛性分析

假设聚合后的梯度

$$g(\mathbf{x}_t; \xi) = \frac{1}{m} \sum_{i=1}^m g(\mathbf{x}; \xi_i) = \frac{1}{mb} \sum_{i=1}^m \sum_{j=1}^b \nabla f(\mathbf{x}; \xi_{i,j})$$

满足以下假设

- **无偏梯度**: 随机梯度估计量 $g(\mathbf{x}; \xi)$ 是对 $\nabla F(\mathbf{x})$ 的无偏估计, 即

$$\mathbb{E}_{\xi}[g(\mathbf{x}; \xi)] = \nabla F(\mathbf{x})$$

- **有界方差**: 随机梯度估计量 $g(\mathbf{x}; \xi)$ 具有有界的方差, 即

$$\text{Var}(g(\mathbf{x}; \xi)) \leq \frac{\sigma^2}{m}$$

$$\mathbb{E}_{\xi}[\|g(\mathbf{x}; \xi)\|^2] \leq \|\nabla F(\mathbf{x})\|^2 + \frac{\sigma^2}{m}.$$

同步随机梯度下降的收敛性分析

同步SGD的收敛性

对于一个 c -强凸和 L -平滑的目标函数，如果学习率 $\eta \leq \frac{1}{L}$ ，并且初始点为 x_0 ，则经过 t 次同步SGD迭代，使用 m 个工作节点和每个工作节点的迷你批量大小 b 时， $F(x_t)$ 的上界为：

$$\mathbb{E}[F(\mathbf{x}_t)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2mc} \leq (1 - \eta c)^t \left(\mathbb{E}[F(\mathbf{x}_0)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2mc} \right)$$

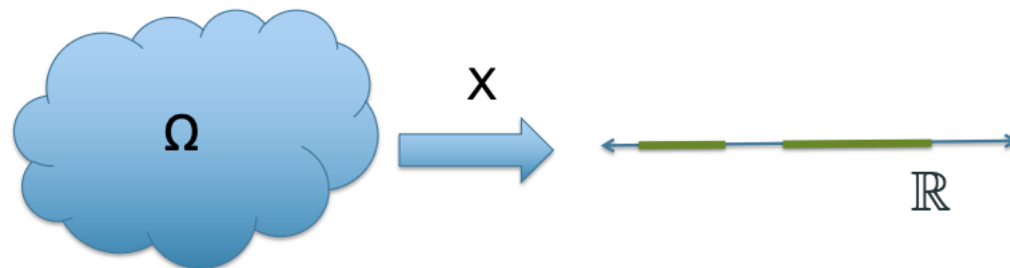
- 对于小批量SGD，当 $t \rightarrow \infty$ 时，将存在一个误差下限
 $\mathbb{E}[F(\mathbf{x}_t)] - F(\mathbf{x}^*) \rightarrow \frac{\eta L \sigma^2}{2mc}$.
- 当增大批次大小 b 时，误差下限降低。
- 当增加工作者数量 m 时，误差下限降低。

增加工作者节点总是更优吗？

**误差随迭代的收敛性随 m 提升，但每次迭代的运行
时如何变化？**

连续型随机变量

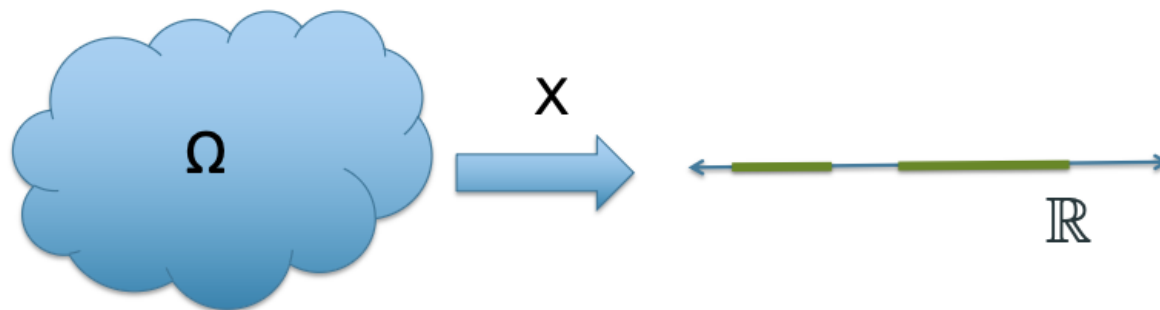
随机变量 X 是将样本空间 Ω 映射到实数轴 $\mathbb{R}$ 上的函数



连续型随机变量的示例

- 指数分布: $f_X(x) = \mu e^{-\mu x}$ for all $x \geq 0$
- 高斯分布: $f_X(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$ for all $x \in \mathbb{R}$

累积与尾部分布函数



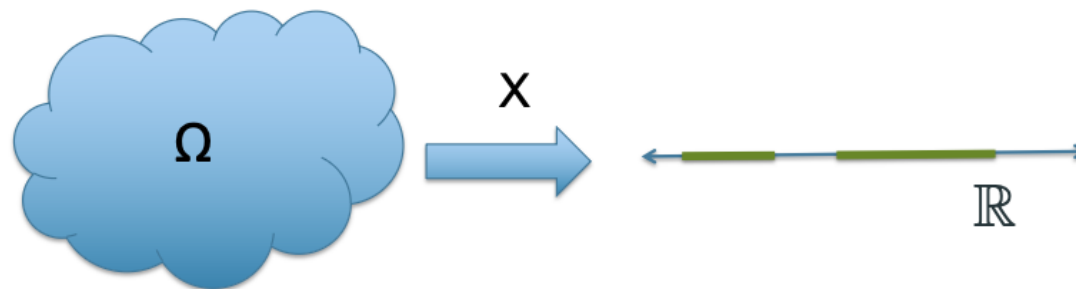
- 累积分布函数 (CDF): $F_X(x) = \Pr(X \leq x)$
- 尾部分布函数 (CCDF): $\bar{F}_X(x) = \Pr(X > x)$

练习：指数分布随机变量的CDF和CCDF是什么？

$X \sim \exp(\mu)$, 当 $x \geq 0$ 时其概率密度函数为 $f_X(x) = \mu e^{-\mu x}$?

- 累积分布函数 (CDF): $F_X(x) = 1 - e^{-\mu x}$ for all $x \geq 0$
- 尾部分布函数 (CCDF): $\bar{F}_X(x) = e^{-\mu x}$ for all $x \geq 0$

期望与方差



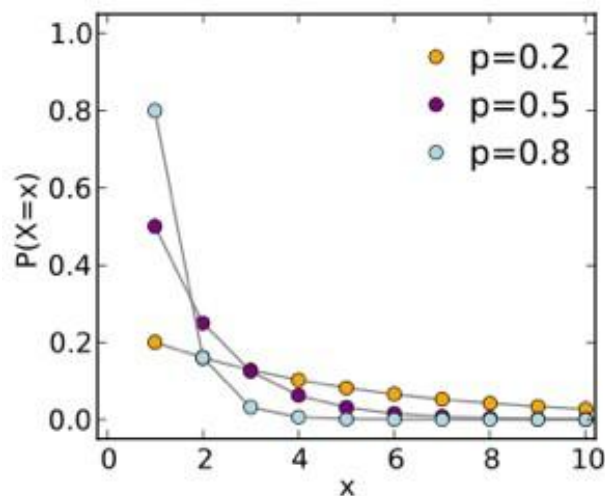
- 期望: $\mathbb{E}[X] = \int_{x \in \Omega} x f_X(x) = \mu_X$
- 对于非负随机变量 $X \geq 0$, 一个等价表达式
关于期望的表达式为: $\mathbb{E}[X] = \int_0^\infty \Pr(X > x) dx$
- 方差: $\text{Var}[X] = \mathbb{E}[(X - \mu_X)^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$

几何随机变量

离散随机变量，其概率质量函数（当 $0 < p < 1$ 时）：

$$\text{概率}(X=k) = (1-p)^{k-1}p \text{ 对于所有 } k=1,2,\dots$$

示例：投掷硬币的次数（硬币有偏向，即正面朝上的概率为 p ），直到首次出现正面为止的情况



指数随机变量

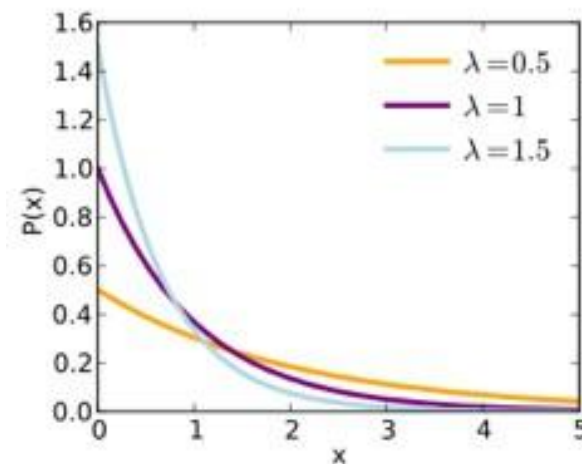
指数分布随机变量的概率密度函数
其速率参数为 λ 时:

$$f_X(x) = \lambda e^{-\lambda x} \quad (x \geq 0)$$

它是几何分布在连续情况下的类比

问题集

- $E[X] = \frac{1}{\lambda}$
- $\text{Var}[X] = \frac{1}{\lambda^2}$
- 无记忆性质: $\Pr(X > x+s | X > s) = \Pr(X > x)$ 对于所有 $x, s \geq 0$



顺序统计量

- 假设有 n 个独立且同分布的随机变量 $X_1, X_2, \dots, X_n$
- 顺序统计量是通过对随机变量进行排序得到的

$$\underbrace{X_{1:n}}_{=\min(X_1, \dots, X_n)} \leq X_{2:n} \leq \dots \leq \underbrace{X_{n:n}}_{=\max(X_1, \dots, X_n)}$$

- $X_{k:n}$ 称为第 k -阶顺序统计量
- 示例：若 $X_1=5, X_2=8$, 且 $X_3=1$, 那么 $X_{2:3}$ 是?

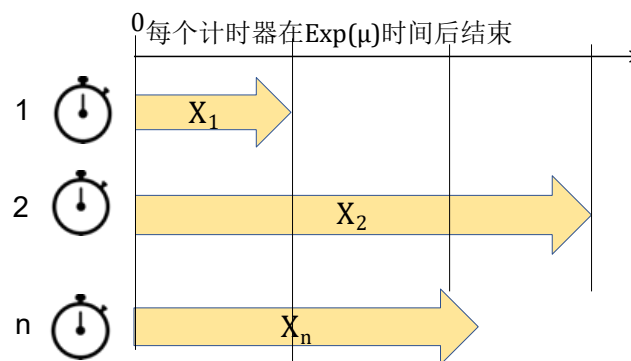
=5

指数分布的顺序统计量

考虑 n 个独立同分布指数随机变量 $X_1, X_2, \dots, X_n$, 其速率 μ 满足

$$f_X(x) = \mu e^{-\mu x} \text{ 对于所有 } x \geq 0$$

- 求 $E[X_{1:n}] = E[\text{最小值}(X_1, \dots, X_n)]$?
- 求 $E[X_{n:n}] = E[\text{最大值}(X_1, \dots, X_n)]$?
- 求 $E[X_{k:n}]$?



指数分布的顺序统计量

考虑 n 个独立同分布指数随机变量 $X_1, X_2, \dots, X_n$, 其速率 μ 满足

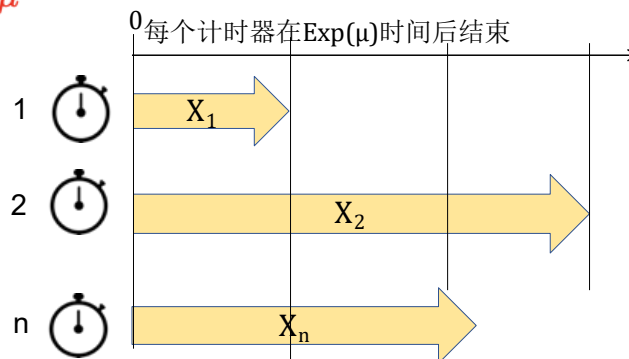
$$f_X(x) = \lambda e^{-\mu x} \text{ 对于所有 } x \geq 0$$

- 求 $E[X_{1:n}] = E[\text{最小值}(X_1, \dots, X_n)]$?

$$\Pr(\text{最小值}(X_1, \dots, X_n) > x) = \Pr(X > x)^n = e^{-n\mu x}$$

因此, 最小值 $(X_1, \dots, X_n)$ 是一个指数随机变量, 其速率 $n\mu$

因此, $E[X_{1:n}] = \frac{1}{n\mu}$



指数分布的顺序统计量

- $E[X_{n:n}] = E[\max(X_1, \dots, X_n)]$?
- 第一个计时器在 $E[X_{1:n}] = \frac{1}{n\mu}$ 时间后超时
- 回忆指数分布的无记忆性特性: $\Pr(X > x+s | X > s) = \Pr(X > x)$ 对所有 $x, s \geq 0$ 成立
- 若一个计时器已运行 s 秒, 其剩余时间仍服从指数分布, 如同刚启动时一样
- 因此, 当第一个计时器超时后, 相当于重新启动剩余的 $n-1$ 个计时器。故,

$$\begin{aligned} E[X_{n:n}] &= E[X_{1:n}] + E[X_{n-1:n-1}] \\ &= \frac{1}{n\mu} + E[X_{n-1:n-1}] \end{aligned}$$

指数分布的顺序统计量

- $E[X_{n:n}] = E[\max(X_1, \dots, X_n)]$?
- 当第一个计时器超时后，相当于重新启动剩余的 $n-1$ 个计时器。故，

$$\begin{aligned} E[X_{n:n}] &= E[X_{1:n}] + E[X_{n-1:n-1}] \\ &= \frac{1}{n\mu} + E[X_{n-1:n-1}] \\ &= \frac{1}{n\mu} + \frac{1}{(n-1)\mu} + E[X_{n-2:n-2}] \\ &= \frac{1}{n\mu} + \frac{1}{(n-1)\mu} + \dots + \frac{1}{\mu} \\ &= \frac{1}{\mu} \left(\frac{1}{n} + \frac{1}{(n-1)} + \dots + \frac{1}{1} \right) \\ &= \frac{H_n}{\mu} \approx \frac{\log n}{\mu} \end{aligned}$$

指数分布的顺序统计量

- 练习：求 $E[X_{k:n}]$ 的值？
- 当第一个计时器超时后，相当于重新启动剩余的 $n-1$ 个计时器。故，

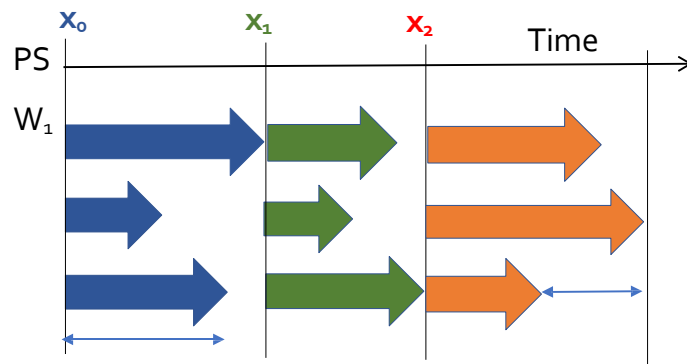
$$\begin{aligned} E[X_{k:n}] &= E[X_{1:n}] + E[X_{k-1:n-1}] \\ &= \frac{1}{n\mu} + E[X_{k-1:n-1}] \\ &= \frac{1}{n\mu} + \frac{1}{(n-1)\mu} + \cdots + \frac{1}{(n-k+1)\mu} \\ &= \frac{H_n - H_{n-k}}{\mu} \approx \frac{1}{\mu} \log \frac{n}{n-k} \end{aligned}$$

现在回到分析同步SGD每次迭代的运行时

同步SGD每次迭代的预期运行时

假设一个工作节点计算并发送其小批量梯度至参数服务器所需时间为随机变量 X

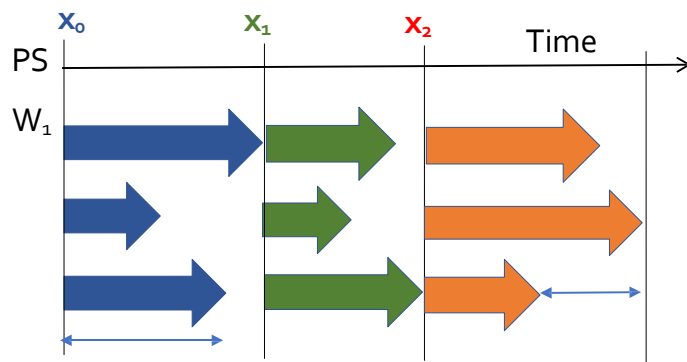
- **问题：**完成一次分布式SGD迭代的预期时间是多少？
- **答案：** $E[X_{m:m}]$, 该值随工作节点数 m 增加而增大
- **问题：** X 的变异性如何影响 $E[X_{m:m}]$? 具体而言, X 的概率分布有何影响?
- **答案：** X 的变异性越高, $E[X_{m:m}]$ 越大。



同步SGD每次迭代的预期运行时

假设一个工作节点计算并发送其小批量梯度至参数服务器的时间服从指数分布 $X \sim \exp(\mu)$

- 问题：一个工作节点计算并发送其梯度的预期时间是多少？
- 答案： $\mathbb{E}[X] = \frac{1}{\mu}$
- 问题：完成一次分布式SGD的预期时间是多少？
- 答案： $\mathbb{E}[X_{m:m}] \sim \frac{\log m}{\mu}$, 其值随工作节点数量 m 的增加而增大



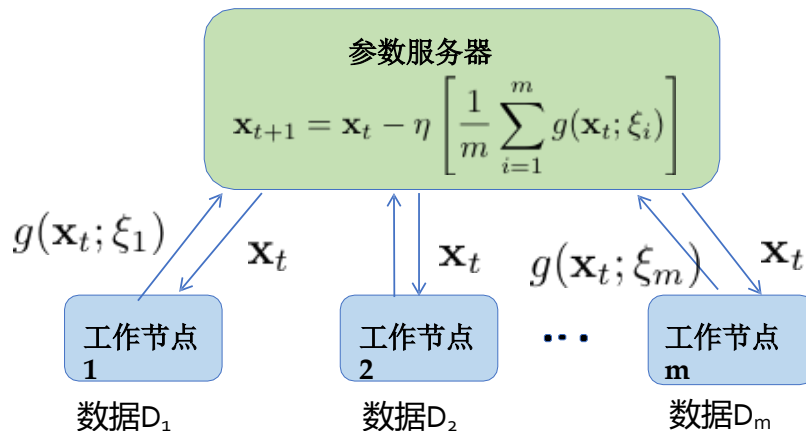
误差随迭代次数收敛的同步SGD

同步SGD的收敛性

对于一个 c -强凸和 L -平滑的目标函数，如果学习率 $\eta \leq \frac{1}{L}$ ，并且初始点为 x_0 ，则经过 t 次同步SGD迭代，使用 m 个工作节点和每个工作节点的迷你批量大小 b 时， $F(x_t)$ 的上界为：

$$\mathbb{E}[F(\mathbf{x}_t)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2mc} \leq (1 - \eta c)^t \left(\mathbb{E}[F(\mathbf{x}_0)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2mc} \right)$$

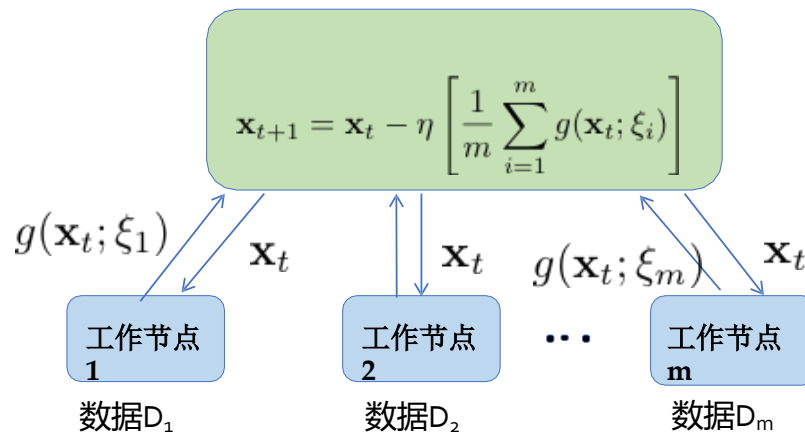
当工作节点 m 增多时，收敛性更佳



同步SGD每次迭代的预期运行时

假设一个工作节点计算并发送其小批量梯度至参数服务器的时间服从指数分布 $X \sim \exp(\mu)$

- 问题：完成一次分布式SGD的预期时间是多少？
- 答案： $\mathbb{E}[X_{m:m}] \sim \frac{\log m}{\mu}$, 其值随工作节点数量 m 的增加而增大



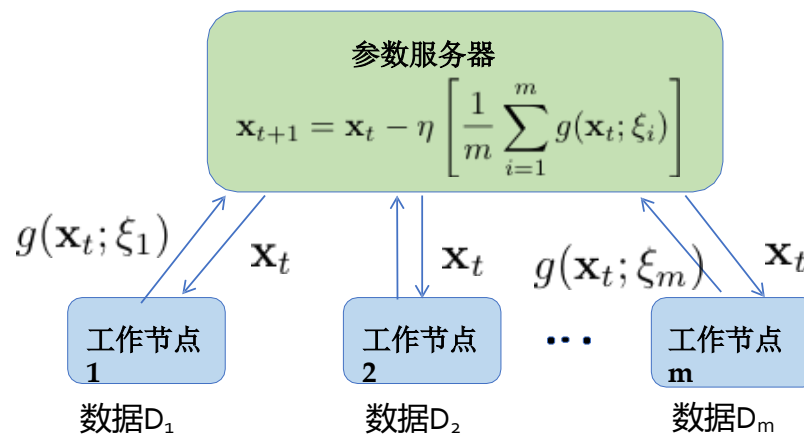
结合误差与迭代次数收敛的结果，可得出误差与实际运行时间的收敛关系

同步SGD的误差与运行时收敛性

同步SGD的收敛性

对于一个 c -强凸和 L -平滑的目标函数，如果学习率 $\eta \leq \frac{1}{L}$ ，并且初始点为 x_0 ，则经过 t 次同步SGD迭代，使用 m 个工作节点和每个工作节点的迷你批量大小 b 时， $F(x_t)$ 的上界为：

$$\mathbb{E}[F(\mathbf{x}_t)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2mc} \leq (1 - \eta c)^t \left(\mathbb{E}[F(\mathbf{x}_0)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2mc} \right)$$



调整工作节点数 m 时，收敛速度与误差下限的权衡

目录

Contents

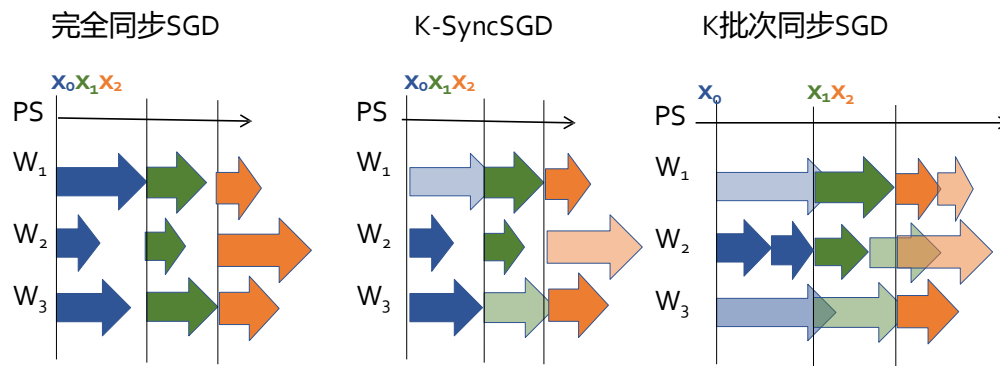
- 1 分布式同步SGD
- 2 抗滞后同步SGD变体
- 3 分布式异步SGD
- 4 模型与流水线并行

K-同步SGD

为避免等待最慢（或滞后）工作节点，可采用以下同步SGD变体：

1. 参数服务器将当前模型版本 $\mathbf{x}_t$ 发送至所有 m 个工作节点
2. 每个工作节点使用大小为 b 的小批次计算梯度 $g(\mathbf{x}_t; \xi_i)$
样本来自数据集 D_i
3. 参数服务器等待直至收到任意 K 个工作节点的梯度
并丢弃其余梯度。利用这 K 个梯度 更新模型 $\mathbf{x}$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{1}{K} \sum_{i=1}^K g(\mathbf{x}_t; \xi_i)$$



但在 $K-1$ 个最快工作节点上仍存在一些空闲时间

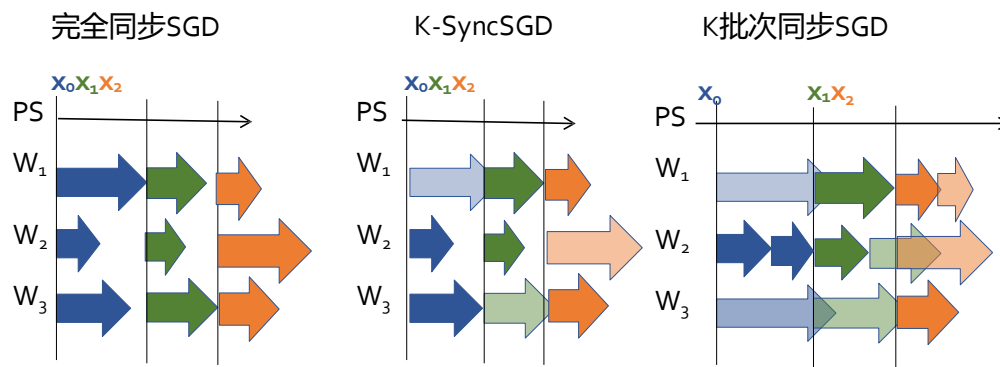
K批次同步SGD

为避免等待最慢（或滞后）工作节点，可采用以下同步SGD变体：

1. 参数服务器将当前模型版本 $\mathbf{x}_t$ 发送至所有 m 个工作节点
2. 每个工作节点持续计算一个或多个小批量梯度使用大小为 b 的小批次计算梯度 $g(\mathbf{x}_t; \xi_i)$ 样本来自数据集 D_i
3. 参数服务器等待直至收到任意 K 个工作节点的梯度

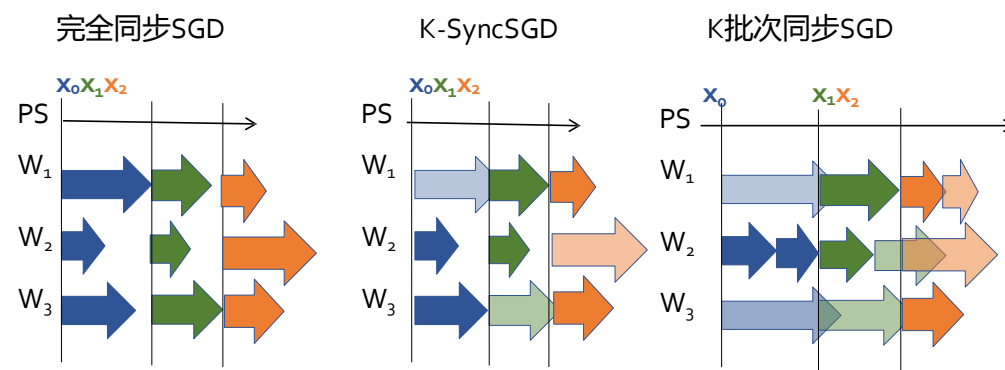
并丢弃其余梯度。利用这 K 个梯度 更新模型 $\mathbf{x}$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{1}{K} \sum_{i=1}^K g(\mathbf{x}_t; \xi_i)$$



K 同步与 K 批次同步SGD的误差收敛

- 更新规则 $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \frac{1}{K} \sum_{i=1}^K g(\mathbf{x}_t; \xi_i)$ 等效于
执行小批量SGD，但批量大小为 Kb 而非仅 b 个样本！
- 因此，小批量SGD的收敛性分析可直接应用于此。
- 唯一变化的参数是方差上界。



K同步与K批次同步SGD的误差收敛

K-同步和K批量同步SGD的收敛性

对于一个 c -强凸和 L -平滑的目标函数，如果学习率 $\eta \leq \frac{1}{L}$ ，并且初始点为 x_0 ，则经过 t 次同步SGD迭代，使用 m 个工作节点和每个工作节点的迷你批量大小 b 时， $F(x_t)$ 的上界为：

$$\mathbb{E}[F(\mathbf{x}_t)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2Kc} \leq (1 - \eta c)^t \left(\mathbb{E}[F(\mathbf{x}_0)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2Kc} \right)$$

- 当 K 减小时，有效小批量尺寸 Kb 随之缩小，导致更高的误差下限

每次迭代的运行时间如何？

K同步与K批次同步SGD的运行时间

K同步随机梯度下降

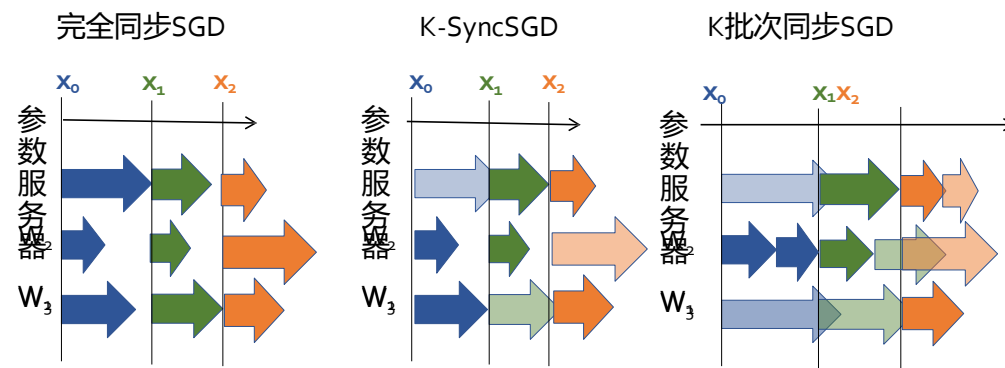
- 假设每个工作节点完成本地梯度计算所需时间 X 服从指数分布(μ)

梯度计算

- 当 m 个工作节点中有 K 个完成其梯度计算时的期望时间为:

$$\mathbb{E}[T] = \mathbb{E}[X_{K:m}] = \frac{H_m - H_{m-K}}{\mu} \sim \frac{1}{\mu} \log \left(\frac{m}{m-K} \right)$$

若 $K=am$ (a 为常数), 则运行时长为恒定值, 而非随 m 增加而增长



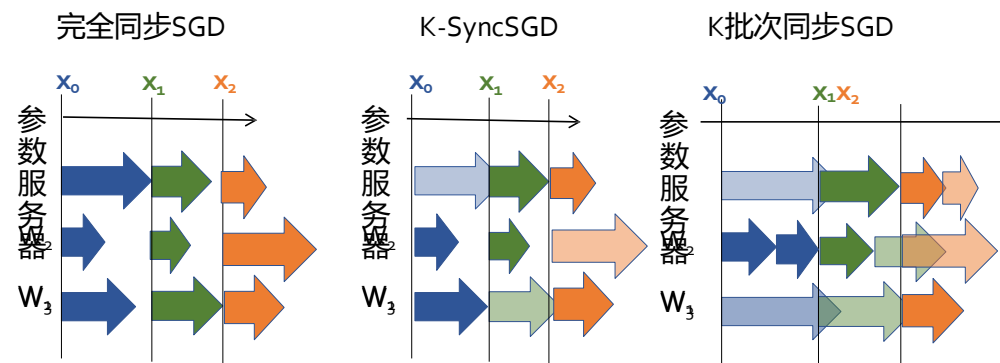
K同步与K批次同步SGD的运行时间

K批量同步随机梯度下降

- 假设每个工作节点完成本地梯度计算所需时间 X 服从指数分布(μ)
- 利用指数分布的无记忆性特性, 参数服务器接收第 i 个与第 $i+1$ 个梯度之间的期望时间, 等同于等待 m 个工作节点中 K 个完成梯度计算的时长
计算期望值: $\mathbb{E}[X_{1:m}] = \frac{1}{m\mu}$
- 因此, 接收 K 个梯度 (快速工作节点可发送多个梯度) 的总时长期望值为:

$$\mathbb{E}[T] = \frac{K}{m\mu}$$

当 K 为常数时, 运行时随工作节点数量 m 增加而降低!



对比所有变体

- 假设每个工作节点完成本地梯度计算所需时间 X 服从指数分布(μ)

$$\mathbb{E}[T_{sync}] = \mathbb{E}[X_{m:m}] \sim \frac{\log m}{\mu}$$

$$\mathbb{E}[T_{K-sync}] = \mathbb{E}[X_{K:m}] \sim \frac{1}{\mu} \log \left(\frac{m}{m-K} \right)$$

$$\mathbb{E}[T_{K-batch-sync}] = \frac{K}{m\mu}$$

- K -批量-同步提供了最佳的运行时扩展性,但它需要更多通信和协调在工作节点与参数服务器之间
- 随着 K 减小,在运行时与误差收敛性之间存在一种权衡
 - 每次迭代的运行时减少
 - 误差下限上升,因为有效的小批量大小 Kb 变得更小

目录

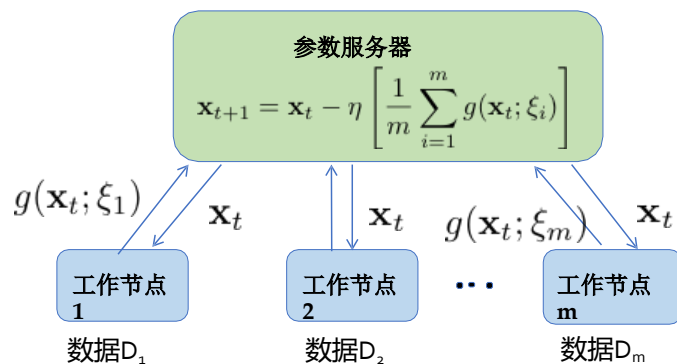
Contents

- 1 分布式同步SGD
- 2 抗滞后同步SGD变体
- 3 分布式异步SGD
- 4 模型与流水线并行

同步随机梯度下降及其变体的缺点

- 工作节点已计算的梯度被丢弃
- 更快的工作节点可能会有空闲时间(除非在K-批量-同步的情况下)

如果完全移除同步屏障并运行异步分布式随机梯度下降会怎样?



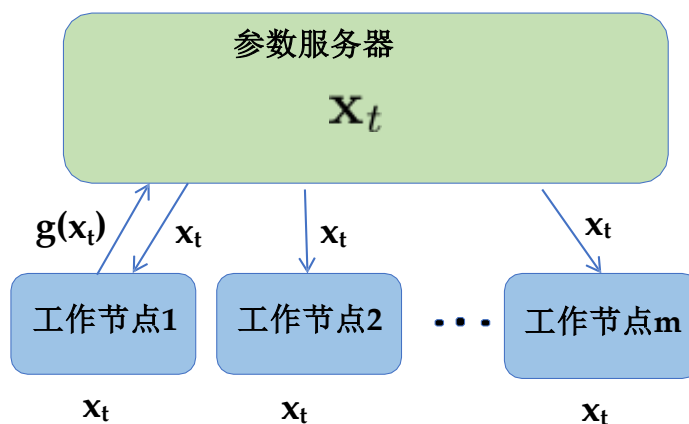
异步分布式随机梯度下降

每个工作节点异步地执行以下操作:

1. 拉取当前模型版本 $\mathbf{x}_t$
2. 计算一个小批量梯度 $g(\mathbf{x}_t)$ 并发送至参数服务器

每当参数服务器接收到来自工作节点的梯度 $g(\mathbf{x}_{\tau(t)})$ 其中 $\tau(t) \leq t$ 时,便更新模型为

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta g(\mathbf{x}_{\tau(t)}; \xi_i)$$



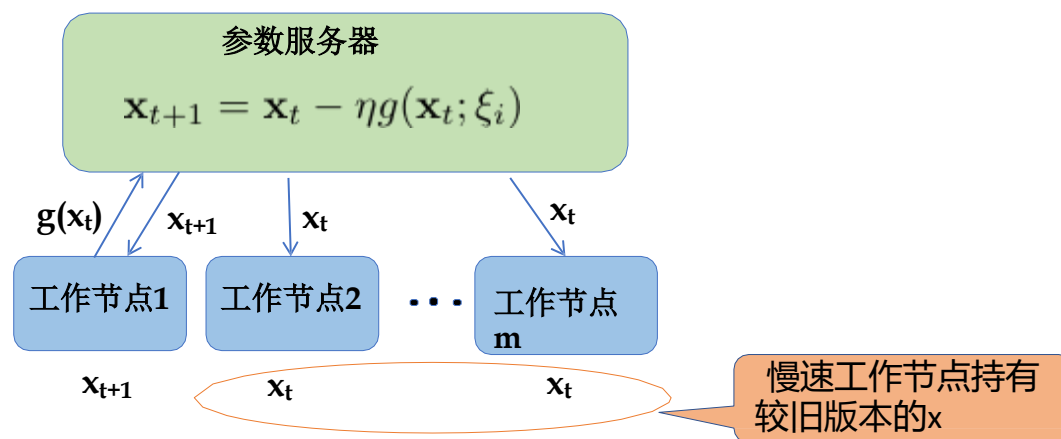
异步分布式随机梯度下降

每个工作节点异步地执行以下操作:

1. 拉取当前模型版本 $\mathbf{x}_t$
2. 计算一个小批量梯度 $g(\mathbf{x}_t)$ 并发送至参数服务器

每当参数服务器接收到来自工作节点的梯度 $g(\mathbf{x}_{\tau(t)})$ 其中 $\tau(t) < t$ 时就会更新模型

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta g(\mathbf{x}_{\tau(t)}; \xi_i)$$



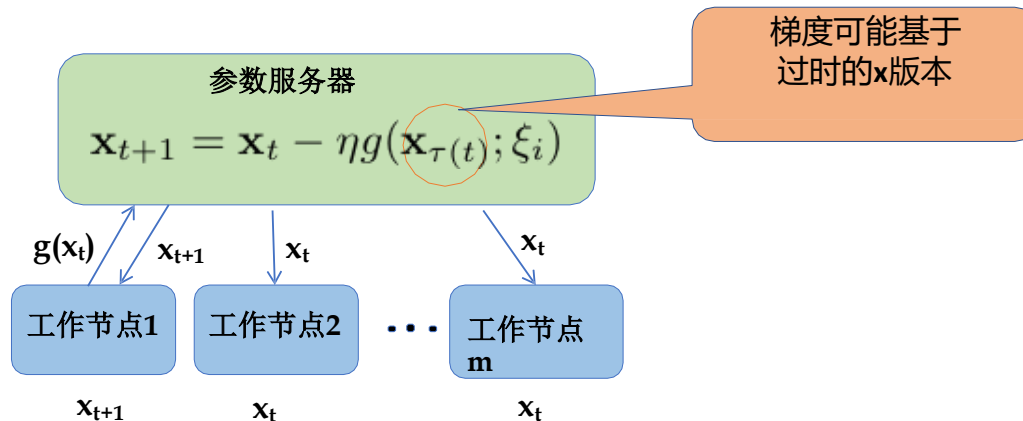
异步分布式随机梯度下降

每个工作节点异步地执行以下操作:

1. 拉取当前模型版本 $\mathbf{x}_t$
2. 计算一个小批量梯度 $g(\mathbf{x}_t)$ 并发送至参数服务器

每当参数服务器接收到来自工作节点的梯度 $g(\mathbf{x}_{\tau(t)})$ 其中 $\tau(t) < t$ 时就会更新模型

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta g(\mathbf{x}_{\tau(t)}; \xi_i)$$

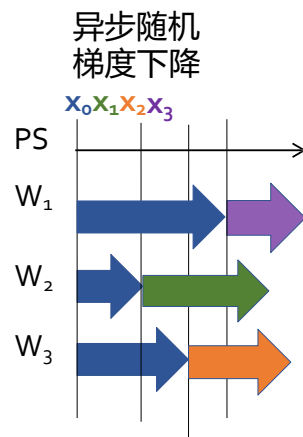


异步随机梯度下降的运行分析

假设每次梯度计算耗时 $X \sim \exp(\mu)$

- 参数向量 $\mathbf{x}$ 会在任意一个工作节点返回梯度时更新
工作节点返回梯度
- 参数服务器接收两个连续梯度之间的期望时间是多少？

$$\begin{aligned}\mathbb{E}[T_{\text{async}}] &= \mathbb{E}[X_{1:m}] \\ &= \frac{1}{m\mu}\end{aligned}$$



异步随机梯度下降的误差分析

关于目标函数的假设

- $F(\mathbf{x})$ 是 L -利普希茨光滑的。这意味着：

$$F(\mathbf{x}) \leq F(\mathbf{y}) + \nabla F(\mathbf{y})^\top (\mathbf{x} - \mathbf{y}) + \frac{L}{2} \|\mathbf{x} - \mathbf{y}\|^2$$

- $F(\mathbf{x})$ 是 c -强凸的。这意味着：

$$F(\mathbf{x}) \geq F(\mathbf{y}) + \nabla F(\mathbf{y})^\top (\mathbf{x} - \mathbf{y}) + \frac{1}{2}c\|\mathbf{x} - \mathbf{y}\|^2 \text{ for all } \mathbf{x}, \mathbf{y} \in \mathbb{R}^d$$

$$2c(F(\mathbf{x}) - F(\mathbf{x}^*)) \leq \|\nabla F(\mathbf{x})\|^2 \text{ for all } \mathbf{x} \in \mathbb{R}^d$$

异步随机梯度下降的误差分析

关于随机梯度的假设

- **无偏梯度**: 随机梯度 $g(\mathbf{x}; \xi)$ 是 $\nabla F(\mathbf{x})$ 的无偏估计, 即:

$$\mathbb{E}_{\xi}[g(\mathbf{x}; \xi)] = \nabla F(\mathbf{x})$$

- **有界方差**: 随机梯度 $g(\mathbf{x}; \xi)$ 具有有界方差, 即:

$$\begin{aligned}\text{Var}(g(\mathbf{x}; \xi)) &\leq \sigma^2 \\ \mathbb{E}_{\xi}[\|g(\mathbf{x}; \xi)\|^2] &\leq \|\nabla F(\mathbf{x})\|^2 + \sigma^2\end{aligned}$$

- **有界陈旧性**: 过时梯度与新鲜梯度的范数平方差被 γ 倍新鲜梯度范数平方上界

$$\mathbb{E}[\|\nabla F(\mathbf{x}_t) - \nabla F(\mathbf{x}_{\tau(t)})\|^2] \leq \gamma \mathbb{E}[\|\nabla F(\mathbf{x}_t)\|^2]$$

较大的 γ 意味着更高的陈旧性

异步随机梯度下降的误差分析

异步SGD的收敛性

对于一个 c -强凸和 L -平滑的目标函数，如果学习率 $\eta \leq \frac{1}{L}$ ，并且初始点为 x_0 ，则经过 t 次同步SGD迭代，使用 m 个工作节点和每个工作节点的迷你批量大小 b 时， $F(x_t)$ 的上界为：

$$\mathbb{E}[F(\mathbf{x}_t)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2\gamma' c} \leq (1 - \eta c \gamma')^t \left(\mathbb{E}[F(\mathbf{x}_0)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2\gamma' c} \right)$$

其中， $\gamma' = 1 - \gamma + \frac{p_0}{2}$ ， γ 是延迟的上界， p_0 是在给定所有过去的延迟和参数的条件下， $\tau(j) = j$ 的条件概率的下界。

- 参数 γ 和 p_0 均取决于梯度计算延迟 X 的概率分布
- 对于指数级梯度计算时间， $p_0 = 1/m$

异步随机梯度下降的误差分析

异步SGD的收敛性

对于一个 c -强凸和 L -平滑的目标函数，如果学习率 $\eta \leq \frac{1}{L}$ ，并且初始点为 x_0 ，则经过 t 次同步SGD迭代，使用 m 个工作节点和每个工作节点的迷你批量大小 b 时， $F(x_t)$ 的上界为：

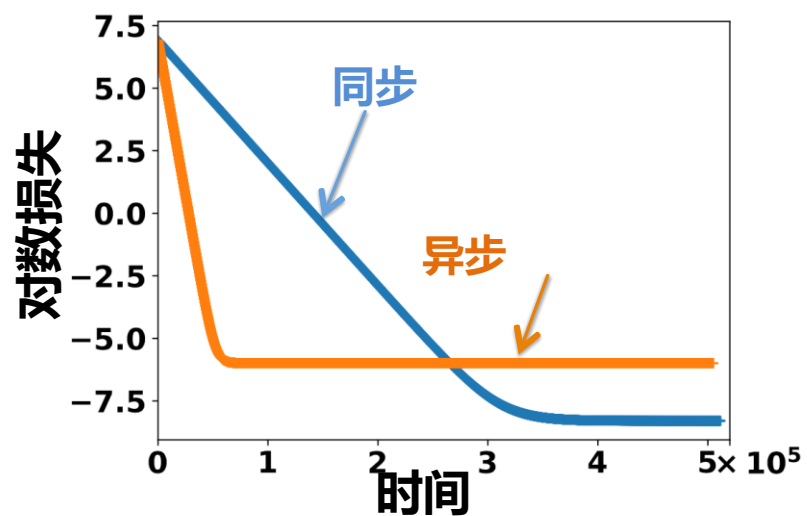
$$\mathbb{E}[F(\mathbf{x}_t)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2\gamma' c} \leq (1 - \eta c \gamma')^t \left(\mathbb{E}[F(\mathbf{x}_0)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2\gamma' c} \right)$$

其中， $\gamma' = 1 - \gamma + \frac{p_0}{2}$ ， γ 是延迟的上界， p_0 是在给定所有过去的延迟和参数的条件下， $\tau(j) = j$ 的条件概率的下界。

- 误差下限 $\frac{\eta L \sigma^2}{2\gamma' c}$ 通常高于同步SGD的误差下限 $\frac{\eta L \sigma^2}{2mc}$
- 当 $p_0/2 > \gamma$ ，收敛速率 $(1 - \eta c \gamma')$ 可快于同步SGD的 $(1 - \eta c)$

同步与异步SGD的误差

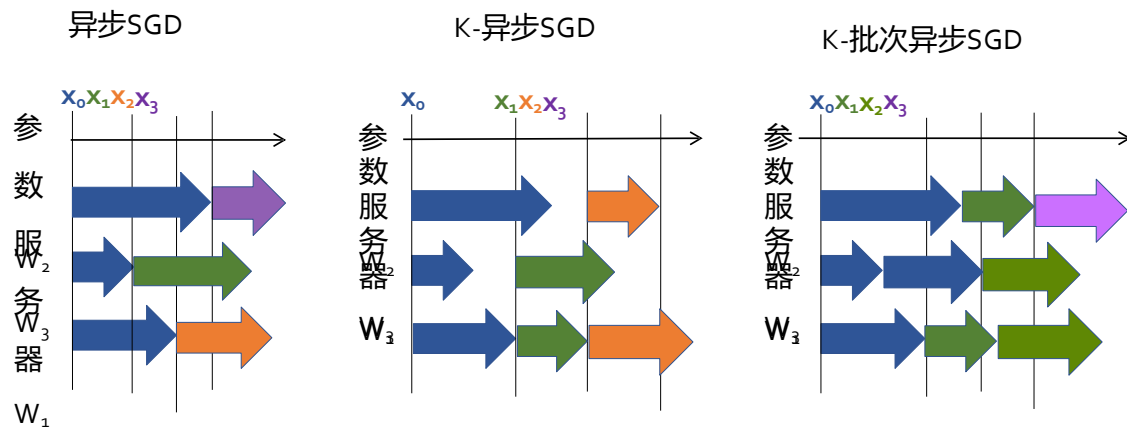
- 异步SGD速度更快，因其单次迭代运行时长远低于同步SGD
- 但异步SGD会达到更高的误差下限



能否在这两种极端情况间找到折中方案？

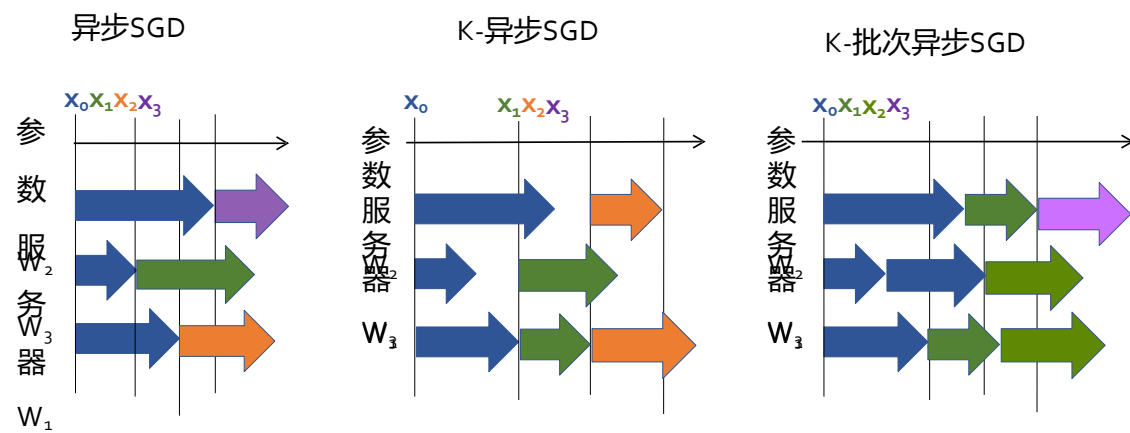
异步SGD的变体

- **K-异步SGD**: 参数服务器等待任意K个工作节点返回梯度后更新 $\mathbf{x}_t$, 并仅向这K个节点发送新参数, 其余节点继续计算陈/旧梯度 $g(\mathbf{x}_t)$
- 与K-同步SGD的区别——不要求慢速节点丢弃已计算的梯度重新开始



异步SGD的变体

- **K-批量异步SGD**: 类似于K-批量同步SGD, 工作节点持续计算梯度在相同的 $\mathbf{x}_t$ 处。参数服务器 (PS) 等待任意K个梯度并利用它们更新 $\mathbf{x}_t$ 。更新后的 $\mathbf{x}_{t+1}$ 会被发送至那些提供梯度的工作节点, 而其他节点继续计算过时的梯度 $g(\mathbf{x}_t)$
- 与K-批量同步SGD的区别——慢速工作节点无需丢弃其梯度计算并重新开始



异步SGD变体的运行时分析

假设每次梯度计算耗时 $X \sim \exp(\mu)$

- **K-异步SGD**: 参数向量 $\mathbf{x}$ 会在任意 K 个工作节点返回其**梯度时更新**。

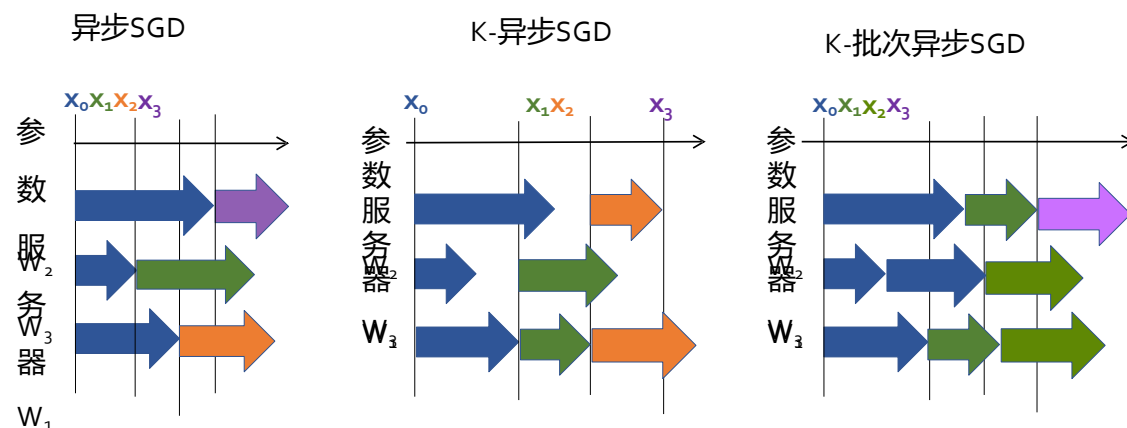
因此,

$$\mathbb{E}[T_{K-async}] = \mathbb{E}[X_{K:m}] = \frac{H_m - H_{m-K}}{\mu} \sim \frac{1}{\mu} \log \left(\frac{m}{m-K} \right)$$

- **K-批量异步SGD**:

参数服务器 (PS) 接收连续梯度的期望时间为 $\mathbb{E}[T_{K-batch-async}] = \frac{1}{m\mu}$

$$\mathbb{E}[T_{K-batch-async}] = \frac{K}{m\mu}$$



异步SGD变体的误差分析

K-异步和K-批量异步SGD的收敛性

对于一个 c -强凸和 L -平滑的目标函数，如果学习率 $\eta \leq \frac{1}{L}$ ，并且初始点为 x_0 ，则经过 t 次异步SGD迭代，使用 m 个工作节点和每个工作节点的迷你批量大小 b 时， $F(x_t)$ 的上界为：

$$\mathbb{E}[F(\mathbf{x}_t)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2\gamma' K c} \leq (1 - \eta c \gamma')^t \left(\mathbb{E}[F(\mathbf{x}_0)] - F(\mathbf{x}^*) - \frac{\eta L \sigma^2}{2\gamma' K c} \right)$$

其中， $\gamma' = 1 - \gamma + \frac{p_0}{2}$ ， γ 是延迟的上界， p_0 是在给定所有过去的延迟和参数的条件下， $\tau(j) = j$ 的条件概率的下界。

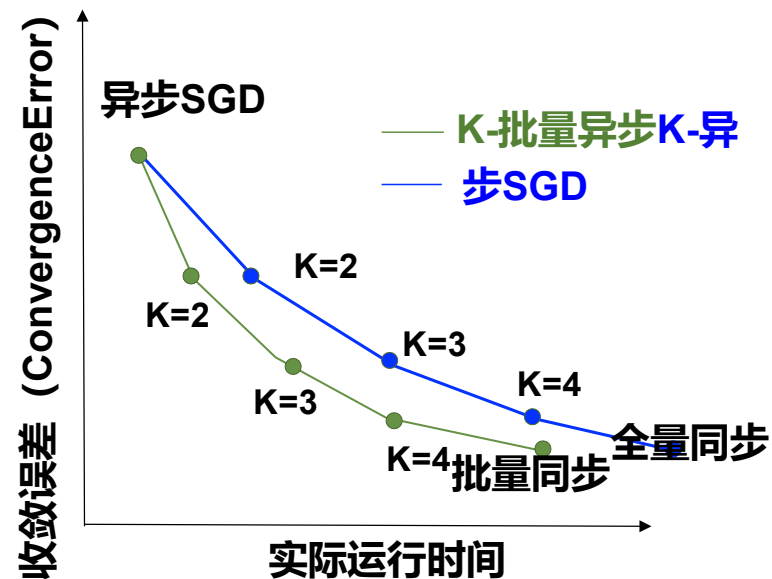
- γ 和 p_0 取决于 K 及采用异步模式还是批量异步SGD

- 误差下限 $\frac{\eta L \sigma^2}{2K\gamma' c}$ 高于同步SGD的误差下限 $\frac{\eta L \sigma^2}{2mc}$

同步与异步SGD变体的对比

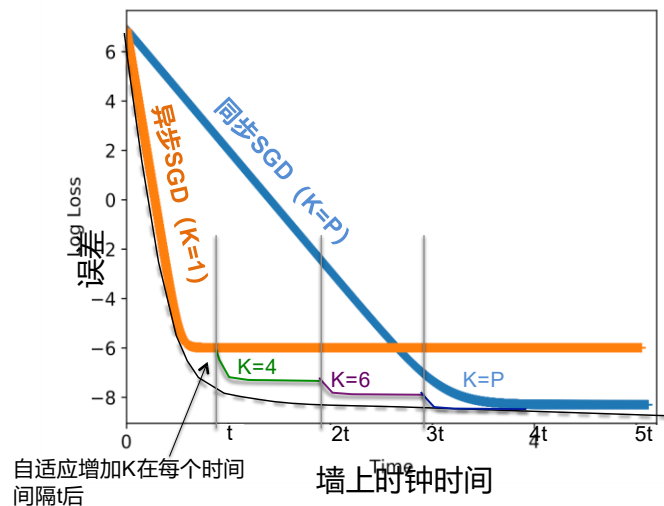
假设每次梯度计算耗时 $X \sim \exp(\mu)$

- X轴—每次迭代的运行时（或实际挂钟时间）
- Y轴—收敛时的误差或误差下限



自适应同步SGD

- 为了实现快速收敛与低误差底线，可以逐步从异步过渡到同步SGD，即从 $K=1$ 开始并逐步增加 K 至 $K=m$
- 这可适用于任何SGD变体： K 同步、 K 批量同步、 K 异步SGD或 K 批量异步SGD

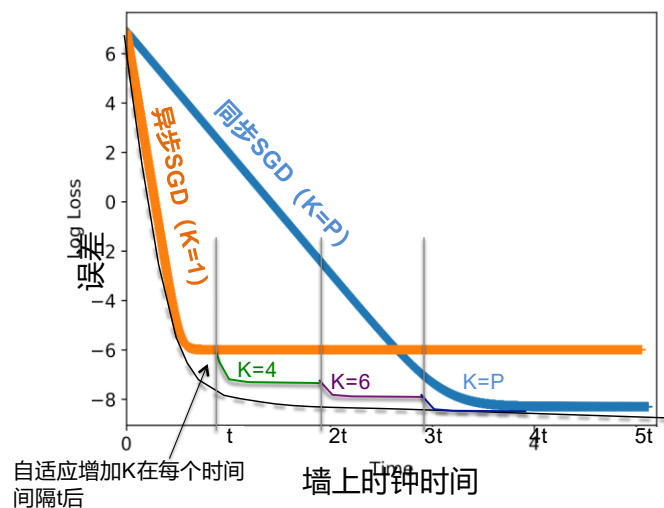


自适应同步SGD

- 为了获得一个启发式调度对于 K 对误差边界在时间 T 处关于 K 求导并设其为零以得到调度方案：

$$K \sim K_0 \sqrt{\frac{F(\mathbf{x}_0)}{F(\mathbf{x}_T)}}$$

其中初始值 K_0 通过网格搜索优化。 $F(\mathbf{x}_T)$ 是目标函数在墙上时钟时间 T 处的值。



自适应同步SGD

- 为了获得一个启发式调度对于 K 对误差边界在时间 T 处关于 K 求导并设其为零以得到调度方案：

$$K \sim K_0 \sqrt{\frac{F(\mathbf{x}_0)}{F(\mathbf{x}_T)}}$$

其中初始值 K_0 通过网格搜索优化。 $F(\mathbf{x}_T)$ 是目标函数在墙上时钟时间 T 处的值。

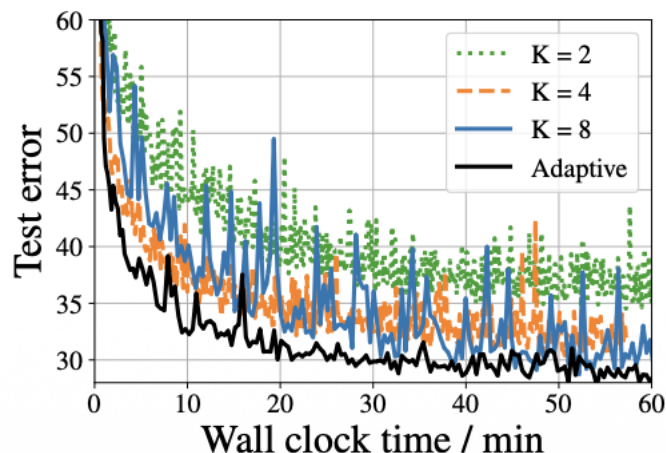


图1：在CIFAR10数据集上使用 $m=8$ 个工作者进行训练

同步/异步SGD在实践中是否被使用？

- 是的！同步/异步SGD是当今工业级机器学习实现的核心
- 最早的工业级实现之一是谷歌的DistBelief框架，详见论文：
https://static.googleusercontent.com/media/research.google.com/en//archive/large_deep_networks_nips2012.pdf.
- DistBelief推广了数据并行和模型并行参数服务器框架。其算法DownpourSGD基于异步SGD和Hogwild

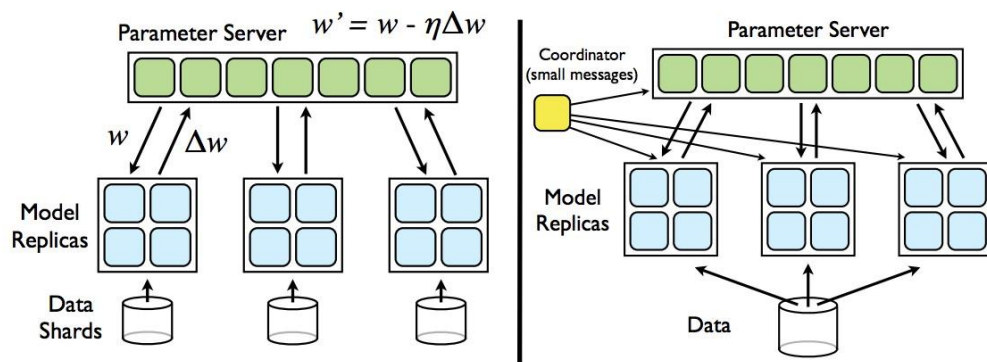


Figure 2: Left: Downpour SGD. Model replicas asynchronously fetch parameters w and push gradients Δw to the parameter server. Right: Sandblaster L-BFGS. A single 'coordinator' sends small messages to replicas and the parameter server to orchestrate batch optimization.

同步/异步SGD在实践中是否被使用？

- 最近，专用硬件如张量处理单元(TPUs)消除了滞后和同步延迟。
- 因此，工业实现正从异步SGD回归到同步SGD
- 但异步SGD在使用廉价消费级硬件和低带宽网络时，仍是一个有用的范式。

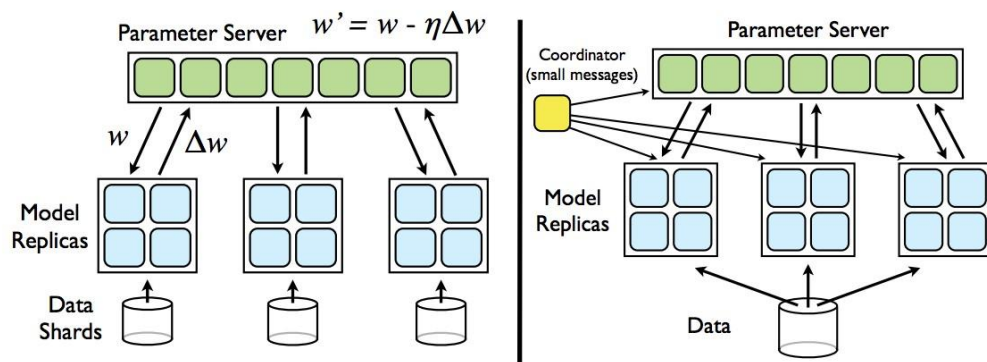


Figure 2: Left: Downpour SGD. Model replicas asynchronously fetch parameters w and push gradients Δw to the parameter server. Right: Sandblaster L-BFGS. A single 'coordinator' sends small messages to replicas and the parameter server to orchestrate batch optimization.

目录

Contents

- 1 分布式同步SGD
- 2 抗滞后同步SGD变体
- 3 分布式异步SGD
- 4 模型与流水线并行**

数据并行

- 目前主要关注数据并行，其中数据在工作者间分区，但每个工作者存储整个模型的当前版本。
- 工作者间参数更新可通过同步或异步方式完成。

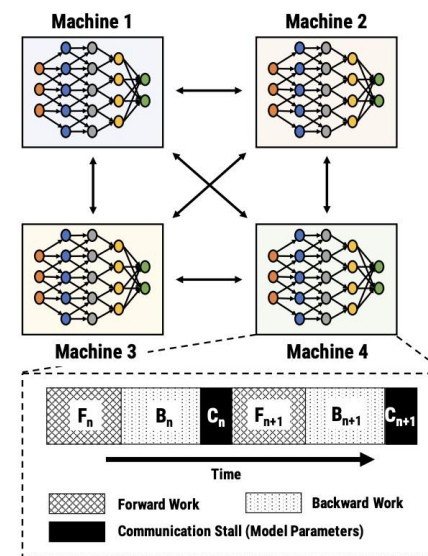


Figure 2: Example data-parallel setup with 4 machines. Timeline at one of the machines shows communication stalls during model parameter exchanges.

数据并行

- 目前主要关注数据并行，其中数据在工作者间分区，但每个工作者存储整个模型的当前版本。
- 工作者间参数更新可通过同步或异步方式完成。
- 大型模型可能无法完全装入单个GPU的内存。

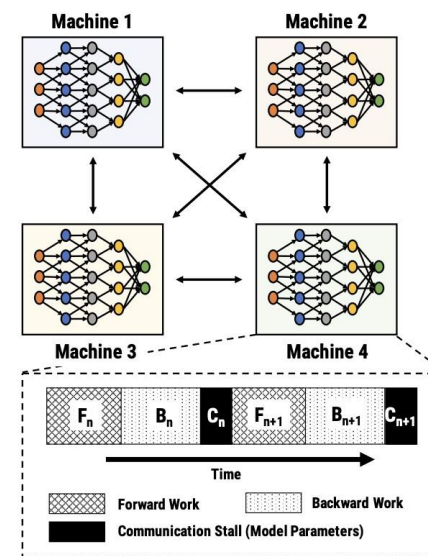


Figure 2: Example data-parallel setup with 4 machines. Timeline at one of the machines shows communication stalls during model parameter exchanges.

模型并行

- 在模型并行中，模型按层拆分到不同GPU上。
- 为避免过时，所有层的前向传播需完成后才能开始反向传播。
- 这导致执行时间线中出现了空闲时间的“气泡”

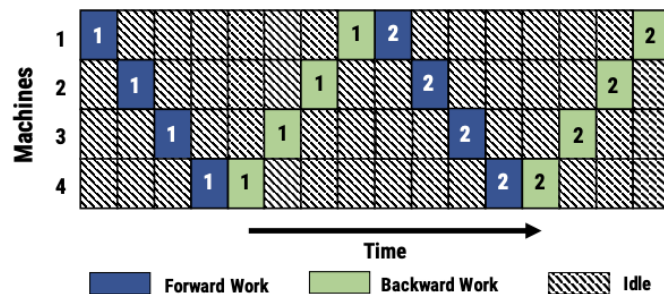
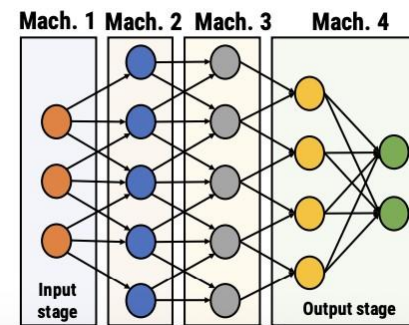


Figure 3: Model parallel training with 4 machines. Numbers indicate minibatch ID. For simplicity, here we assume that forward and backward work in every stage takes one time unit, and communicating activations across machines has no overhead.

流水线并行

- 结合数据并行与模型并行训练——一台机器可以在上一小批量的反向传播完成前，就开始下一小批量的前向传播过程。
 - 减少流水线中空闲时间的“气泡”
 - 每台机器需要一个调度算法来优先处理队列中的任务。下方时间线展示的是一种1前向1反向调度器
- 1前向1反向调度器
- 前向传播与反向传播所采用的模型版本之间可能存在滞后现象

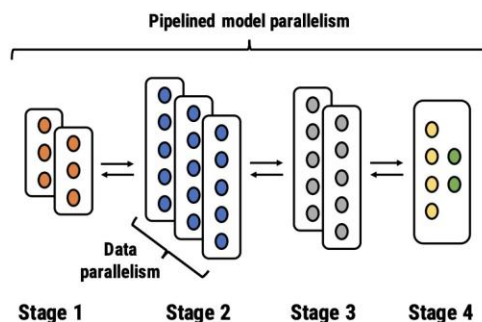


Figure 6: Pipeline Parallel training in PipeDream combines pipelining, model- and data-parallel training.

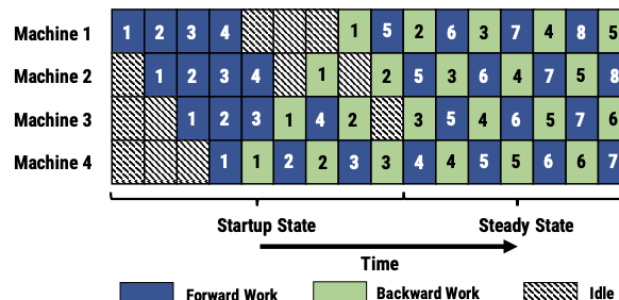


Figure 8: An example pipeline with 4 machines, showing startup and steady states.

参考资料

- 缓慢与滞后的梯度也能赢得比赛：
<https://arxiv.org/abs/1803.01113>
- 异步随机梯度下降的锐利收敛性分析：
<https://arxiv.org/abs/2206.08307>
- 异步并行随机梯度下降：<https://arxiv.org/abs/1506.08272>
- Pipedream：<https://arxiv.org/pdf/1806.03377>